

Subconsultas (Subqueries) en SQL y su Aplicación en la Obtención y Modificación de Datos

Esta lectura aborda el dominio de subconsultas en SQL como herramienta fundamental para resolver requerimientos complejos de filtrado, reglas de negocio, auditoría y depuración de calidad de datos en entornos profesionales.

Qué se espera dominar y por qué importa en la práctica

El resultado de aprendizaje de este módulo exige que puedas diagnosticar cuándo conviene anidar sentencias y que implementes subqueries tanto para consulta como para modificación de datos, resolviendo requerimientos reales (filtrado, reglas de negocio, auditoría, depuración de calidad, etc.). El programa explicita que debes reconocer casos de uso de subqueries y construirlas como herramienta para obtención y modificación de datos. Ese desempeño se apoya en competencias previas del mismo módulo (consultas simples, agrupaciones, agregaciones, filtros, y DML: INSERT/UPDATE/DELETE).

Expresión escalar

Devuelve un solo valor integrada en SELECT, WHERE/HAVING, o listas de asignación UPDATE/INSERT

Conjunto de filas

Unidimensional o tabular, consumido con IN/ANY/ALL, EXISTS, o en FROM como "tabla derivada"

CTE (Common Table Expression)

Con WITH, ofrece subqueries "nombradas" reutilizables, incluso recursivas

Además, el optimizador re-escribe (pull-up) muchas subqueries a planes equivalentes con JOIN cuando es seguro hacerlo, mejorando rendimiento sin cambiar el resultado. Esto es relevante para decidir entre "expresividad" y "eficiencia" sin caer en micro-optimización prematura.

Consultas anidadas: conceptos nucleares

Tipologías de subqueries

Escalares

Devuelven 1 valor (numérico, fecha, texto). Útiles para columnas calculadas, umbrales dinámicos o asignaciones en DML.

Multifila

Devuelven una lista; se consumen con IN, ANY, ALL.

Multicolumna

Devuelven una tabla virtual, típicamente en FROM (derivada). Requieren alias.

Correlacionadas

La subquery hace referencia a columnas de la consulta externa; se "evalúa" conceptualmente por fila de la externa.

Dónde pueden ubicarse

En SELECT, FROM (derivadas), WHERE, HAVING, y también dentro de DML: INSERT, UPDATE, DELETE (como filtro, como fuente de valores o en RETURNING de CTEs).

Por qué subqueries (y no sólo JOINS)



Legibilidad y encapsulación

Expresan pasos lógicos (cálculo intermedio → condición) sin ensuciar el FROM.



Alineación con la lógica del negocio

"Primero obtén el tope/patrones, luego decide".



Menos tablas temporales

Evitas staging en esquemas intermedios.



Modificación condicional

UPDATE/DELETE con EXISTS/IN o UPDATE ... FROM (extensión muy usada en PostgreSQL).

Industria: en retail se comparan precios de productos vs. promedios por categoría; en banca se promueven clientes con saldo por encima del percentile target; en healthcare se purgan eventos huérfanos; en telecom/IoT se cruzan lecturas "outliers" vs. baseline dinámico; en ciberseguridad se hace "semi/anti-join" para detectar entidades sin evidencia asociada (EXISTS/NOT EXISTS).

Proceso de evaluación de subqueries

Cómo "piensa" el motor

01

Evaluación de la subquery

O la integra al plan global si la eleva/colapsa.

02

Propagación del resultado

A la consulta externa como un valor, una lista o un conjunto tabular.

03

Resolución de la consulta externa

Filtro, proyección, modificación.

PostgreSQL puede elevar subqueries (pull-up), materializarlas (en CTEs), o fusionarlas con el plan si NOT MATERIALIZED lo permite y no hay efectos laterales. En selección entre subquery vs. JOIN, recuerda que el optimizador suele converger a un plan similar si la equivalencia semántica lo permite.



Implicación práctica: escribe primero la versión más clara. Optimiza con EXPLAIN sólo cuando el costo sea real (datos grandes, picos de latencia, SLAs). Planificación y pruning a menudo te evitan "tocar" el SQL si el modelo está bien indexado.

Reglas de implementación

- Alias obligatorio en derivadas (FROM (subquery) AS t), porque la tabla "no tiene nombre" por sí misma.
- Cardinalidad: una subquery escalar debe devolver exactamente una fila y una columna; si devuelve 0 filas, en PostgreSQL el resultado es NULL (útil con COALESCE); si devuelve >1 fila, error.
- Semántica de IN/ANY/ALL:
 - IN compara a un conjunto (equivalente a = ANY(subquery));
 - ANY/SOME prueban "verdad para algún elemento";
 - ALL exige "verdad para todos".
- EXISTS/NOT EXISTS: semántica booleana; el motor puede "parar" en la primera coincidencia (eficiente con índices).
- Correlación: cada fila de la externa "parametriza" la subquery; evita cálculos redundantes (o muévelos a CTE) si son invariantes.
- CTE con WITH:
 - Nombra subqueries reutilizables (mejor lectura);
 - En PostgreSQL puede materializar por defecto (un solo cómputo, consistente) o fusionar con NOT MATERIALIZED;
 - WITH RECURSIVE habilita topologías jerárquicas (árboles, grafos);
 - CTE modificadoras (INSERT/UPDATE/DELETE ... RETURNING) generan una "tabla temporal lógica" consumible por la consulta principal.

Cuándo es recomendable utilizar subqueries

Umbral dinámico

Promedios, percentiles, top-N dependiente del propio dataset (no un literal).

Filtro semántico

Con existencia (EXISTS) o inexistencia (NOT EXISTS) de hechos en otras tablas.

Auditoría/bitácora

INSERTs selectivos desde eventos vivos; también en CTE con RETURNING.

Purga y saneamiento

Borrar huérfanos (DELETE ... WHERE NOT EXISTS (...)).

Upserts diferidos

UPDATE solo para filas que superan cierto cálculo de referencia.

Derivadas con agregaciones

Para simplificar HAVING o para encapsular KPIs.

Casos industriales

- **Retail:** reposición de stock únicamente en ítems "top-decile" de rotación (subquery con percentil).
- **Marketing:** activar campañas para clientes con LTV > media por cohorte (correlacionada).
- **Banca/Finanzas:** DELETE de operaciones duplicadas sin clearing relacionado (NOT EXISTS).
- **Salud:** UPDATE de estados clínicos cuya última medición excede un z-score por grupo (derivada + correlación).
- **Manufactura:** INSERT en bitácoras de calidad cuando defectos > p95 de la línea (CTE + RETURNING).

Subqueries en SELECT (proyección calculada)

Escalares para enriquecer filas

```
SELECT  
c.id_cliente,  
c.nombre,  
(SELECT COUNT(*) FROM pedidos p WHERE p.id_cliente = c.id_cliente) AS pedidos_totales  
FROM clientes c;
```

Uso: dashboards operacionales; métricas por entidad (clientes, proveedores, activos).

Importancia: evita JOIN + GROUP BY si sólo necesitas el número; la intención es cristalina.

Coste: con millones de filas podría ser costoso; si el conteo es global por cliente y lo vas a usar varias veces, dévalo en CTE o materialízalo.

Derivadas en FROM para KPIs por grupo

```
SELECT d.categoria, d.promedio_precio  
FROM (  
  SELECT categoria, AVG(precio) AS promedio_precio  
  FROM productos  
  GROUP BY categoria  
) AS d;
```

Uso: indicadores por jerarquía; pre-agregados consumidos por múltiples vistas.

Importancia: aislamiento del cálculo (mantenibilidad); la consulta externa puede hacer JOIN o filtros adicionales.

Nota: el optimizador puede levantar la derivada si conviene.

Subqueries en INSERT (cargar desde consultas)

Insertar registros "vivos" que cumplen condición

```
INSERT INTO empleados_historial (id, nombre, depto, salario, fecha)
SELECT id, nombre, depto, salario, CURRENT_DATE
FROM empleados
WHERE depto = 'Ventas';
```

Uso: snapshot histórico, auditorías, migraciones selectivas.

Importancia: sin staging; atomicidad y reproducibilidad.

Thresholds de percentil (caso real de HR/Comp)

```
INSERT INTO auditoria_sueldos (id, sueldo, marca, ts)
SELECT e.id, e.sueldo, 'p90', CURRENT_TIMESTAMP
FROM empleados e
WHERE e.sueldo > (
  SELECT PERCENTILE_CONT(0.9) WITHIN GROUP (ORDER BY sueldo)
  FROM empleados
);
```

Uso: monitoreo de dispersiones salariales (equidad).

Importancia: cumplimiento y trazabilidad; el umbral nace de los datos presentes (no un fijo).

❑ **CTE modificadora + RETURNING:** en PostgreSQL, una CTE de INSERT/UPDATE/DELETE con RETURNING expone una tabla lógica consumible por la query principal. Es ideal para auditar lo que se insertó/actualizó y encadenarlo con otra acción.

Subqueries en UPDATE (asignaciones y filtros dinámicos)

Filtro por referencia correlacionada

```
UPDATE empleados e
SET sueldo = sueldo * 1.05
WHERE e.sueldo < (
  SELECT AVG(sueldo) FROM empleados WHERE depto = e.depto
);
```

Uso: ajustes por debajo de la media de la unidad.

Importancia: políticas de compensación alineadas a comparables internos.

UPDATE ... FROM (extensión habitual en PostgreSQL)

```
UPDATE t1
SET b = t2.b
FROM t2
WHERE t1.a = t2.a;
```

El árbol de ejecución es muy similar a un JOIN, y el optimizador trata ambos casos como planes equivalentes.

Uso: sincronizar atributos desde una tabla maestra; "propagar" cambios calculados en una derivada intermedia.

Subqueries en DELETE (depuración y saneamiento)

Anti-join de huérfanos

```
DELETE FROM pedidos p
WHERE NOT EXISTS (
  SELECT 1 FROM clientes c WHERE c.id_cliente = p.id_cliente
);
```

Uso: purgar registros sin integridad referencial (cuando no hay FK o en entornos temporales).

Importancia: calidad y consistencia de datos.

Rendimiento: EXISTS suele ser eficiente con índices (para en la primera coincidencia).

Eliminación condicional por topología de actividad

```
DELETE FROM sesiones s
WHERE s.estado = 'CERRADA'
AND s.ts_cierre < (
  SELECT MAX(ts_cierre)
  FROM sesiones
  WHERE usuario = s.usuario
) - INTERVAL '180 days';
```

Uso: políticas de retención (compliance/privacidad).

Importancia: minimiza riesgo y costo de almacenamiento.

EXISTS, IN, ANY, ALL: cuándo elegir cada uno



EXISTS (y NOT EXISTS)

Responde "¿existe al menos una fila relacionada?"; óptimo para condiciones de existencia/inexistencia y anti-joins.



IN

Sirve para listas enumeradas o conjuntos retornados por subquery; es legible cuando el valor comparado es la clave.



ANY/SOME vs ALL

Útiles con comparadores distintos de $=$; p. ej., " $> ANY(subquery)$ " (mayor que alguno), " $> ALL(subquery)$ " (mayor que todos).



Escalares

Elige subquery escalar si conceptualmente es un valor global (percentil, media, máximo calificado) que alimenta muchas filas.

Casos industriales:

- EXISTS para "clientes con al menos un ticket abierto".
- NOT EXISTS para "productos sin ficha de atributos" (data quality).
- $> ALL$ para "precios por sobre toda la competencia en la categoría".

CTE (WITH) vs. subquery directa

Cuándo WITH:

- Reutilizas el mismo resultado en múltiples lugares
- Quieres nombrar pasos lógicos
- Consideras recursión (jerarquías)
- Encadenas DML con RETURNING para auditar lo afectado

Materialización:

Por defecto, una CTE se evalúa una sola vez (garantía de consistencia y costos controlados). Con NOT MATERIALIZED, el optimizador puede fusionarla con la query principal (ahorro si consumes pocas filas y hay oportunidades de empuje de predicados).

Rendimiento y planificación

- **Índices en columnas de unión**

Y filtros de subqueries (EXISTS/IN) reducen I/O.

- **EXPLAIN**

Inspecciona si hay Nested Loop vs Hash/Merge Join, escaneos indexados y cardinalidades estimadas.

- **Particionado**

Prunes eficientemente particiones irrelevantes; a veces no precisarás reescrituras; el plan ya evita escaneos innecesarios.

- **Evita correlaciones innecesarias**

Si el valor no depende de la fila, súbelo a CTE o subquery no correlacionada.

- **Cuidado con subqueries escalares**

Que pueden devolver >1 fila: valida cardinalidad (o usa agregación MAX/AVG).

Comparación con ventanas analíticas

Muchas tareas de "comparar contra el promedio/percentil del grupo" pueden implementarse con ventanas:

```
SELECT  
p.*,  
AVG(precio) OVER (PARTITION BY categoria) AS avg_cat  
FROM productos p;
```

Ventanas no colapsan el resultado (no requieren GROUP BY) y suelen rendir mejor que una subquery correlacionada repetitiva. Úsalas cuando sólo necesitas el valor junto a cada fila y no un filtro que dependa de él. (El resultado del aprendizaje AE4 sigue exigiendo subqueries, pero en la práctica conocer ambas técnicas te hace más eficiente).

Ejercicio 1: SELECT con subquery correlacionada

Precio > promedio de su categoría

Concepto: subquery correlacionada en WHERE; cálculo por grupo dinámico.

Uso: catálogo; detección de outliers de precio por categoría.

Importancia: pricing governance; decisiones de margen.

```
SELECT p.id_producto, p.nombre, p.categoria, p.precio
FROM productos p
WHERE p.precio > (
  SELECT AVG(p2.precio)
  FROM productos p2
  WHERE p2.categoria = p.categoria
);
```

Variación con ventana (cuando solo quieres exhibir el KPI):

```
SELECT p.*,
  AVG(precio) OVER (PARTITION BY categoria) AS avg_cat
FROM productos p;
```


Ejercicio 2: INSERT con umbral de percentil

Top-10% de salarios

Concepto: subquery escalar con percentil; INSERT ... SELECT.

Uso: auditoría/comp & ben; retención de talento.

Importancia: compliance/equidad; detección de dispersiones.

```
INSERT INTO empleados_auditoria (id_empleado, nombre, salario, fecha_auditoria)
SELECT id, nombre, salario, CURRENT_DATE
FROM empleados
WHERE salario > (
  SELECT PERCENTILE_CONT(0.9) WITHIN GROUP (ORDER BY salario)
  FROM empleados
);
```

CTE con RETURNING para encadenar notificación (PostgreSQL):

```
WITH ins AS (
  INSERT INTO empleados_auditoria (id_empleado, nombre, salario, fecha_auditoria)
  SELECT id, nombre, salario, CURRENT_DATE
  FROM empleados
  WHERE salario > (
    SELECT PERCENTILE_CONT(0.9) WITHIN GROUP (ORDER BY salario)
    FROM empleados
  )
  RETURNING id_empleado
)
INSERT INTO notificaciones (id_empleado, canal, mensaje, ts)
SELECT id_empleado, 'email', 'Ingresó a auditoría p90', NOW()
FROM ins;
```

Ejercicio 3: UPDATE de stock por ventas superiores al promedio

Concepto: subquery tabular + agregaciones; anti-pattern de doble agregación resuelto con derivada.

Uso: supply; recorte o reasignación de inventario.

Importancia: capital de trabajo; rotación saludable.

```
UPDATE productos
SET stock = stock * 0.9
WHERE id_producto IN (
  SELECT id_producto
  FROM ventas
  GROUP BY id_producto
  HAVING SUM(cantidad) >
  (SELECT AVG(total_ventas)
   FROM (SELECT SUM(cantidad) AS total_ventas
        FROM ventas
        GROUP BY id_producto) v)
);
```

Alternativa UPDATE ... FROM (PostgreSQL): más claro y suele planificarse igual que un JOIN.

```
WITH ventas_por_prod AS (
  SELECT id_producto, SUM(cantidad) AS tot
  FROM ventas
  GROUP BY id_producto
),
prom AS (
  SELECT AVG(tot) AS promedio FROM ventas_por_prod
)
UPDATE productos p
SET stock = p.stock * 0.9
FROM ventas_por_prod v, prom
WHERE p.id_producto = v.id_producto AND v.tot > prom.promedio;
```

Ejercicio 4: DELETE de pedidos cancelados sin cliente

Anti-join de huérfanos

Concepto: NOT EXISTS (anti-join).

Uso: limpieza tras migraciones; calidad de datos.

Importancia: consistencia referencial cuando no hay FK (o es temporal).

```
DELETE FROM pedidos p
WHERE p.estado = 'Cancelado'
AND NOT EXISTS (
  SELECT 1
  FROM clientes c
  WHERE c.id_cliente = p.id_cliente
);
```

Ejercicios adicionales de casos industriales



Clientes activos con tickets abiertos

EXISTS para CRM/soporte; priorización de cuentas



Candidatos marketing

Consolidación de reglas; segmentación



Salario por cohorte

Correlacionada; justicia interna

Consejos prácticos y alineación con el programa

Consejos que maximizan el valor en la industria

	Empieza por lo legible El optimizador suele convertir subqueries y derivadas en planes de join equivalentes.
	Valida cardinalidades Si esperas 1 fila y podrían existir varias, agrega MAX() o LIMIT 1 controlado (con orden definido).
	Indexa para existencia Índices en llaves usadas por EXISTS/IN marcan la diferencia en latencia.
	Usa CTE para auditar CTEs modificadoras con RETURNING capturan exactamente lo que tocaste.
	Explora ventanas Si sólo necesitas "comparar con el grupo", una ventana puede ser más clara/eficiente.
	UPDATE ... FROM Para sincronizaciones masivas; su plan es casi igual que un JOIN de lectura.
	EXPLAIN temprano Si los datos son grandes o la consulta será crítica para un SLA.
	Retención y privacidad Subqueries en DELETE ayudan a cumplir ventanas de retención y "derecho al olvido".

Alineación con el programa y cierre

- El AE4 del Módulo 6 exige construir sentencias anidadas para obtener y modificar datos y reconocer cuándo convienen; así se consigna y evalúa en el plan.
- La competencia del módulo pone foco en construir consultas SQL que satisfacen requerimientos informacionales (insights), y en el mismo módulo se especifica el dominio de consultas simples, agregaciones y DML que sirven de base a este AE.
- En términos de estándar y comportamiento del motor, PostgreSQL reconoce a la subconsulta escalar como expresión de valor, formaliza WITH/RECURSIVE (incluidas CTEs modificadoras con RETURNING) y detalla cómo materializa o fusiona CTEs, además de elevar subqueries cuando es óptimo.

El resultado: subqueries son un instrumento central para traducir reglas de negocio complejas a SQL claro, auditable y, con un mínimo de disciplina (índices, EXPLAIN, CTEs), escalable. Su valor en la industria es transversal: mejor gobernanza de datos, eficiencia operativa, cumplimiento normativo y rapidez para convertir requerimientos en consultas y cambios correctos sobre la base de datos.